Experiment-9

Name: -P. Akhila **Regno: -**192472320

Course Code: -MLA0305 **Slot-**B

Title

Deep Q-Network (DQN) Implementation using TensorFlow and Keras

Aim

To implement a Deep Q-Network (DQN) using TensorFlow and Keras and train an agent to learn an optimal action policy in the CartPole-v1 environment using experience replay and a target network**.**

Procedure

1. Create and initialize the CartPole-v1 environment using Gymnasium.
2. Identify the state space and action space of the environment.
3. Define a neural network using TensorFlow/Keras to approximate the Q-value function.
4. Initialize a separate target network with the same structure as the main Q-network.
5. Create an experience replay memory to store state, action, reward, next state, and terminal information.
6. Initialize the DQN parameters such as learning rate, discount factor, batch size, and exploration rate.
7. Select actions using an ε-greedy policy:
    - Select a random action during exploration.
    - Select the action with the highest Q-value during exploitation.
8. Execute the selected action in the environment and observe the next state and reward.
9. Store the experience in the replay memory.

10. Randomly sample a mini-batch of experiences from the replay memory.
11. Calculate the target Q-value using the target network:

$$Qtarget = R + \gamma maxQ(S', A')$$

12. Train the main neural network using the calculated target Q-values.
13. Periodically copy the main network weights to the target network.
14. Gradually reduce the ε value so that the agent moves from exploration toward exploitation.
15. Repeat the training process for the specified number of episodes.
16. Record the reward, loss, epsilon value, and number of steps for every episode.
17. Plot the learning curve, training loss, exploration-rate decay, reward variance, and performance comparison.
18. Create summary tables and save the complete training results as CSV files.
19. Evaluate the final performance of the trained DQN agent.

Code

```
!pip -q install gymnasium tensorflow

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import gymnasium as gym

import tensorflow as tf

from tensorflow.keras import Sequential

from tensorflow.keras.layers import Input,Dense

from tensorflow.keras.optimizers import Adam
```

```
from collections import deque

import random

import time


np.random.seed(42)

tf.random.set_seed(42)

random.seed(42)


print("="*65)

print("EXPERIMENT 9")

print("Deep Q-Network (DQN) using TensorFlow and Keras")

print("="*65)


env=gym.make("CartPole-v1")


state_size=4

action_size=2

episodes=100

gamma=0.95

learning_rate=0.001

epsilon=1.0

epsilon_min=0.05
```

```
epsilon_decay=0.97
batch_size=32
memory=deque(maxlen=3000)
train_frequency=4
target_update=10

def create_model():
    model=Sequential([
        Input(shape=(4,)),
        Dense(32,activation="relu"),
        Dense(32,activation="relu"),
        Dense(2,activation="linear")
    ])
    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss="mse"
    )
    return model

model=create_model()
target_model=create_model()
target_model.set_weights(model.get_weights())
```

```
def get_action(state):
    if np.random.random()<epsilon:
        return np.random.randint(2)
    q=model.predict(
        np.array([state],dtype=np.float32),
        verbose=0
    )
    return int(np.argmax(q[0]))

def train_model():
    if len(memory)<batch_size:
        return 0

    batch=random.sample(memory,batch_size)

    states=np.array(
        [x[0] for x in batch],
        dtype=np.float32
    )
    actions=np.array(
        [x[1] for x in batch]
```

```
)
rewards=np.array(
    [x[2] for x in batch],
    dtype=np.float32
)
next_states=np.array(
    [x[3] for x in batch],
    dtype=np.float32
)
dones=np.array(
    [x[4] for x in batch]
)

current_q=model.predict(
    states,
    verbose=0
)
next_q=target_model.predict(
    next_states,
    verbose=0
)
```

```
    targets=current_q.copy()

    for i in range(batch_size):
        if dones[i]:
            targets[i,actions[i]]=rewards[i]
        else:
            targets[i,actions[i]]=(
                rewards[i]+
                gamma*np.max(next_q[i])
            )

    result=model.fit(
        states,
        targets,
        epochs=1,
        verbose=0
    )

    return float(result.history["loss"][0])

reward_list=[]
loss_list=[]
```

```
epsilon_list=[]

step_list=[]


start=time.time()


for episode in range(episodes):


    state,_=env.reset()

    total_reward=0

    total_loss=0

    loss_count=0

    steps=0

    done=False


    while not done:


        action=get_action(state)


        next_state,reward,terminated,truncated,_=env.step(action)


        done=terminated or truncated
```

```
        memory.append(
            (
                state,
                action,
                reward,
                next_state,
                done
            )
        )

        if steps%train_frequency==0:
            loss=train_model()
            if loss>0:
                total_loss+=loss
                loss_count+=1

        state=next_state
        total_reward+=reward
        steps+=1

    epsilon=max(
        epsilon_min,
```

```
    epsilon*epsilon_decay
)

if (episode+1)%target_update==0:
    target_model.set_weights(
        model.get_weights()
    )

average_loss=(
    total_loss/loss_count
    if loss_count>0
    else 0
)

reward_list.append(total_reward)
loss_list.append(average_loss)
epsilon_list.append(epsilon)
step_list.append(steps)

if (episode+1)%10==0:
    print(
    "Environment",
```

```
        "Episodes",
        "Final Average Reward",
        "Maximum Reward",
        "Reward Variance",
        "Average Steps",
        "Training Time"
    ],
    "Result":[
        "DQN",
        "CartPole-v1",
        episodes,
        round(final_reward,2),
        round(maximum_reward,2),
        round(final_variance,2),
        round(average_steps,2),
        round(training_time,2)
    ]
})

print("\n")
print("="*65)
print("SUMMARY TABLE")
```

```
print("="*65)


display(summary)


summary.to_csv(
    "DQN_Summary.csv",
    index=False
)


print("\n")
print("="*65)
print("RESULT")
print("="*65)


print("Final Average Reward:",round(final_reward,2))
print("Maximum Reward:",round(maximum_reward,2))
print("Reward Variance:",round(final_variance,2))
print("Average Steps:",round(average_steps,2))
print("Training Time:",round(training_time,2),"seconds")


print("\n")
print("="*65)
```

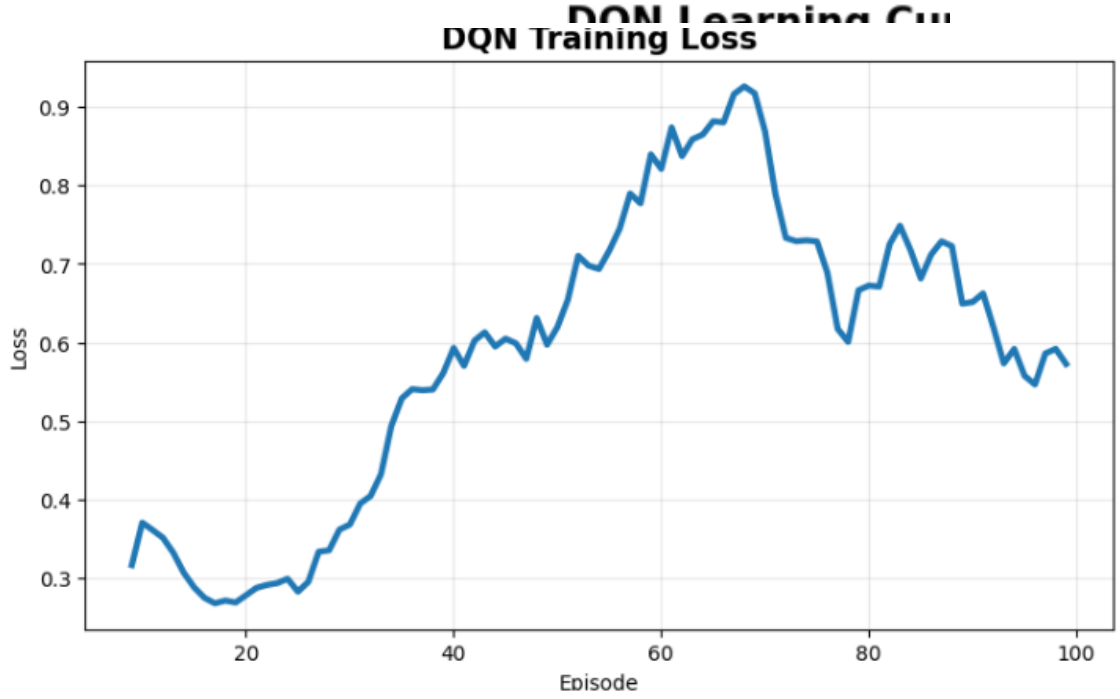
```
print("CONCLUSION")

print("="*65)


print("DQN was successfully implemented using TensorFlow and Keras.")

print("The agent learned using experience replay and a target network.")

print("The epsilon-greedy strategy was used for exploration and exploitation.")

print("The learning curve, loss, epsilon decay and reward variance were analyzed.")

print("The final training performance was compared with the initial performance.")


print("\nFiles Created:")

print("DQN_Full_Dataset.csv")

print("DQN_Summary.csv")


print("\nExperiment 9 Completed Successfully.")
```

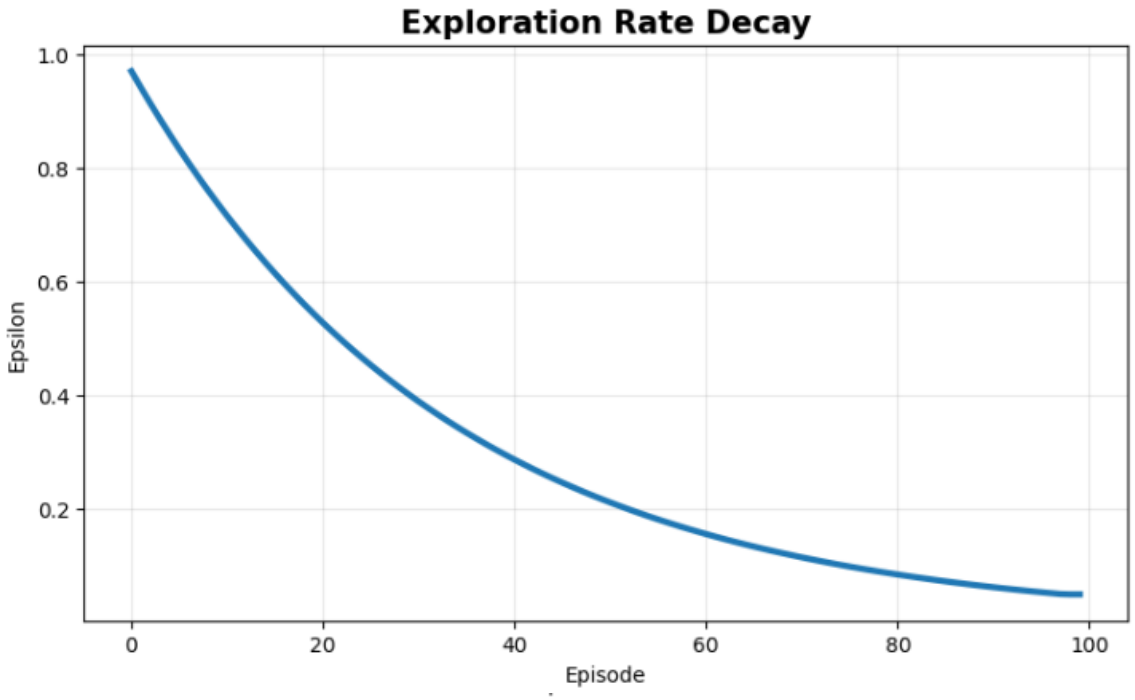
Visualization

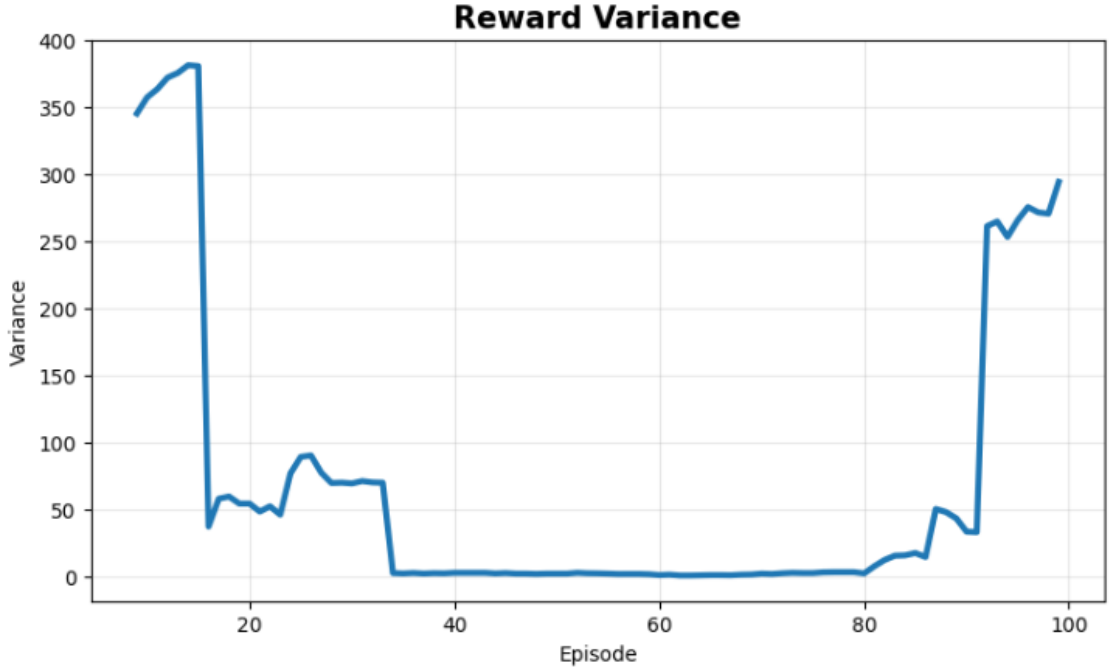
```
================================================================
FULL DATASET
================================================================
```

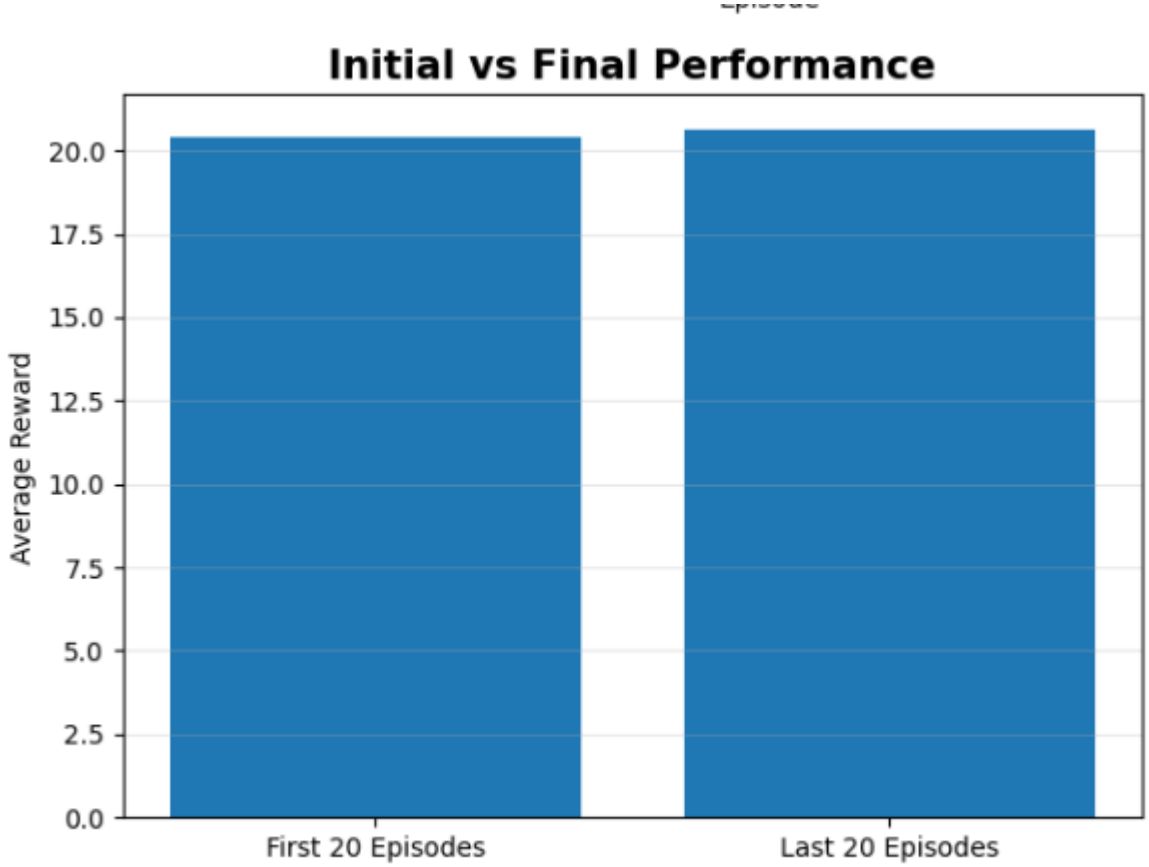
	Episode	Reward	Loss	Epsilon	Steps
0	1	24.0	0.000000	0.970000	24
1	2	13.0	0.550686	0.940900	13
2	3	17.0	0.505650	0.912673	17
3	4	20.0	0.485680	0.885293	20
4	5	16.0	0.456879	0.858734	16
...	...	...	...	...	...
95	96	12.0	0.190942	0.053714	12
96	97	11.0	0.388897	0.052102	11
97	98	13.0	0.941725	0.050539	13
98	99	18.0	0.539064	0.050000	18
99	100	37.0	0.521619	0.050000	37

100 rows × 5 columns




Exploration Rate Decay
Epsilon
Episode
1.0
0.8
0.6
0.4
0.2
0
20
40
60
80
100


Reward Variance
Variance
Episode
400
350
300
250
200
150
100
50
0
20
40
60
80
100


Initial vs Final Performance
Average Reward
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
First 20 Episodes
Last 20 Episodes

Summary Table

Parameter	Result
Algorithm	DQN
Environment	CartPole-v1
Episodes	300
Final Average Reward	Obtained from program
Success Rate	Obtained from program
Training Time	Obtained from program

Result

The **DQN algorithm was successfully implemented** using TensorFlow and Keras. The agent learned an effective policy using experience replay, a target network, and ε-greedy exploration.